

# Assignment 2 - Part 2 - COMP2152

## Part 2 - Explain your Code (50%)

Write the answers in the doc provided and save and submit Part 2 as a PDF. You do not have to write or submit any new code for this section. I want you to understand how you could work on a piece of code that already exists (as is the case when working with Open Source code), and how to improve it. You can **type** in your answers or complete it **by hand** (handwriting MUST be legible) and then scan your submission.

1. How have we used classes for our project to reuse code?  
We have used classes to reuse code by having the Character class contain the shared properties (combat\_strength and health\_points) as well as the constructor and destructor that can then be inherited by the Hero and Monster classes. This way the code only needs to be written in one place (instead of repeating the same/similar code in both the Hero and Monster class), this is more efficient and easier to maintain.
2. Provide 1 line of code, as one of many examples, where code is shared between the monster class and the hero class?  
`self.combat_strength = functions.small_dice_roll(small_dice_options)`
3. What is the benefit of using complex getters and setters?  
Complex getters allow for calculations or transformation of the data taken from an attribute before returning a value.  
Complex setters allow for logic to validate the data being set for a property, making sure it follows the “rules” of the class.
4. If we didn't use try-except blocks, what would be the problem?  
Try-except blocks allow us to control what happens when exceptions occur, so that the program can continue to run instead of crashing.  
Try-except blocks also let us specify the error message that gets displayed to the user, and/or allow us to log the errors for debugging purposes.
5. How could we use the name of the **operating system** or the **version of python** in your game to prevent errors? Choose just 1 of the above.  
We could implement logic based on the name of the operating system to prevent errors by taking into account OS-specific differences, e.g. how file paths or line breaks are represented. Making sure the right version of this is used for each system to help prevent crashes or unexpected behaviours.  
File paths - windows: \ and case-insensitive, unix: / and case-sensitive  
Line breaks - windows: \r\n, unix: \n

6. What's another piece of information we could save inside of the save.txt file?  
(Remember, we load this information every time we start a new game, so that we can keep track of all of the games you have played so far. )  
We could save the weapon the hero was assigned and/or the loot collected in each game, and/or the power the monster had in the game.  
Alternatively, we could save the inception dream level selected in each game.

## 7. New Feature:

- a. Think of **1 new feature** you can add to the game that could use list comprehension and nested conditional statements. For now just write 1 sentence that describes the feature:

Now add your new feature description here:

The Hero comes across the door to a science lab with a Crazy Scientist inside that attempts to clone either the monster or the hero which affects their combat\_strength or health\_points in various ways.

---

### Examples:

Below are the examples to show you that you can be very creative, and you should have fun with this exercise. You must use an idea that is NOT directly on the list below:

**eg a)** Add another monster so that the hero can fight 2 monsters at once

**eg b)** Create a digital board game, that shows the hero moving around to different towns on a map

**eg c)** Add a dog that runs in front of the hero and discovers features about the world

- 
- b. Give the new feature you created a short 2-3 -word a title:

Now write your Title here:

Crazy Scientist

---

### Examples:

**eg a)** Multiple Monsters

**eg b)** Roam Towns

**eg c)** Dog Scout

---

- c. Explain how you could implement the idea you chose. You must explain how you would use both of the control structures below. Draw a diagram, map, sketch for each (you can use any software for this, e.g. Draw.io). You don't have to match the style of diagram I have here, just use a visual to describe your idea. Note, you must have loops and conditional statements diagrammed below as needed:

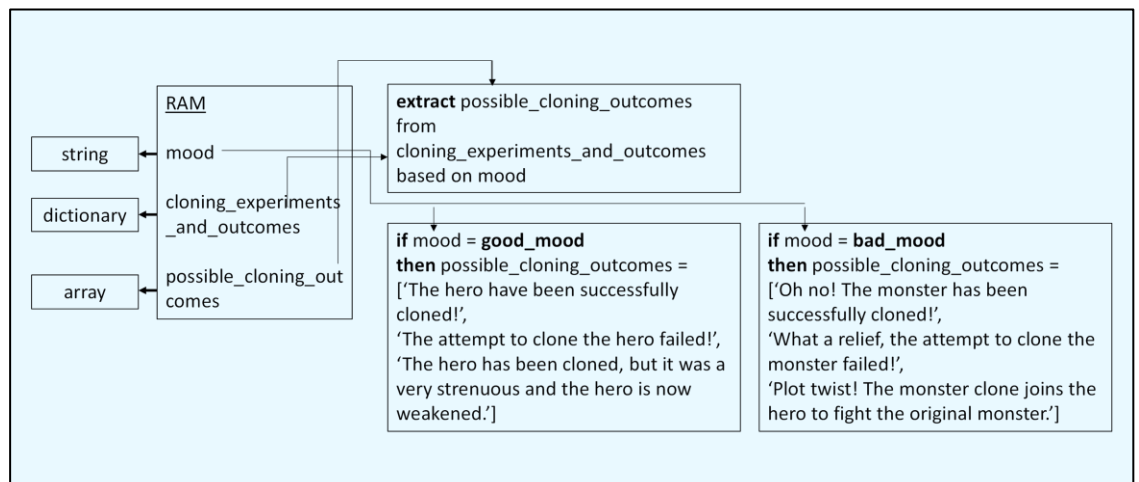
i. **Using a list comprehension loop**

The variable mood would be set to either good\_mood or bad\_mood (set in the nested conditional statements). This variable would then be used as the conditional in the list comprehension loop to iterate thorough the dictionary of cloning options and extract a new list containing only the options that match the mood.

```
cloning_experiments_and_outcomes = {  
'good_mood' : ['The hero have been successfully cloned!', 'The attempt to  
clone the hero failed!', 'The hero has been cloned, but it was a very strenuous  
and the hero is now weakened.'],  
'bad_mood' : ['Oh no! The monster has been successfully cloned!', 'What a  
relief, the attempt to clone the monster failed!', 'Plot twist! The monster clone  
joins the hero to fight the original monster.']  
}
```

```
possible_cloning_outcomes = [value for key, value in  
cloning_experiments_and_outcomes.items() if key == mood]
```

This new list would then be used in the game where a small dice roll would determine the outcome of the cloning experiment, this would in turn affect combat\_strength and/or health\_points of the hero and/or monster character.



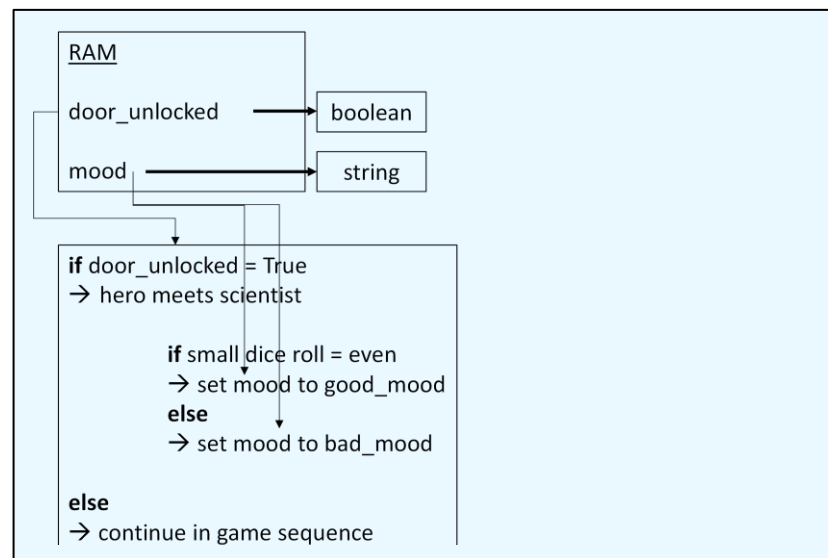
ii. **Using nested conditional statements**

This would be used to first determine **if** the door to the science lab is unlocked. Unlocked – **then** Hero meets the Crazy Scientist, **else** Locked – Hero moves on in the game sequence. A big dice roll would determine the number rolled and the variable door\_unlocked would be set accordingly. 1-16 – True (Unlocked), 17-10 – False (Locked).

Second, **if** the Hero meets the Crazy Scientist, **then** a small dice roll would determine if the Scientist is in a bad mood (odd roll) or a good mood (even roll) and save this result into the variable mood.

Bad mood – “Evil experiments” – Clone the monster.

Good mood – “Helpful experiments” – Clone the hero.



---

**Example:**

**eg b) Roam Towns**

i. **Using a list comprehension loop**

Every time in the loop, move one square in 1 direction, (N, E, S, W). Have a variable that keeps track of the Hero's location by saving values of the board. We can have 2 nested for loops and store the map as a 2D array.

Eg.

Hero location is currently at Row 3, Column D.

Town 2 location is at Row 4, Column G.  
 Town 1 location is at Row 1, Column A .

**Diagram:**

|   | A           | B | C | D            | E | F | G           | H | I |
|---|-------------|---|---|--------------|---|---|-------------|---|---|
| 1 | (town1_loc) |   |   |              |   |   |             |   |   |
| 2 |             |   |   |              |   |   |             |   |   |
| 3 |             |   |   | (player_loc) |   |   |             |   |   |
| 4 |             |   |   |              |   |   | (town2_loc) |   |   |

ii. **Using a nested Conditional Statement**

**If** the hero is in Town 2, **then** allow the hero to buy armor but not sell.  
 Otherwise, the hero can sell armor but cannot buy.

Create an array of armor options available in Town 2. He could also trade some of his loot based on the value of the loot he has.

